

David_数论

David_数论

排列组合
数论基础
数论进阶
线性基与矩阵

排列组合

```
1  const int mod = 1e9 + 7;
2  const int maxn = 2e5;
3
4
5  //模逆元 排列数
6  vector<int> fac(maxn + 5), inv(maxn + 5);
7  int ksm(int x, int a, int mod){
8      int ans = 1;
9      while(a){
10         if(a & 1) ans = ans * x % mod;
11         x = x * x % mod;
12         a >>= 1;
13     }
14     return ans;
15 }
16
17 void init(){
18     fac[0] = 1;
19     for(int i = 1; i <= maxn; i++){
20         fac[i] = fac[i - 1] * i % mod;
21     }
22     inv[maxn] = ksm(fac[maxn], mod - 2, mod);
23     for(int i = maxn - 1; i >= 0; i--){
24         inv[i] = inv[i + 1] * (i + 1) % mod;
25     }
26 }
27
28 int A(int n, int m){
29     return fac[n] * inv[n - m] % mod;
30 }
31
32 int C(int n, int m){
33     return fac[n] * inv[m] % mod * inv[n - m] % mod;
34 }
35
36 //错位排列Derangement
37 int Derangement(int n){
38     vector<int> dp(n + 1);
39     dp[1] = 0;
40     dp[2] = 1;
41     for(int i = 3; i <= n; i++){
42         dp[i] = (n - 1) * (dp[i - 1] + dp[i - 2]) % mod;
43     }
```

```

44 }
45
46 //卡特兰数CatalanNumber 解决不交叉问题的种类数
47 //有 n 个左括号与 n 个右括号。能组成多少个合法括号序列?
48 int CatalanNumber(int n){
49     int ans = C(2 * n, n) - C(2 * n, n + 1);
50     return (ans % mod + mod) % mod;
51 }

```

数论基础

```

1 //欧拉筛
2 const int mod = 1e9 + 7;
3 const int maxn = 1e6;
4 vector<int> primes;
5 //欧拉筛 同时 预处理最小质因数 与 目标数分解质因数
6
7 int min_primes[maxn + 5];
8 void init(){
9     for(int i = 2; i <= maxn; i++){
10         if(!min_primes[i]){
11             min_primes[i] = i;
12             primes.push_back(i);
13         }
14         for(int p : primes){
15             if(p > min_primes[i] || i * p > maxn) break;
16             min_primes[i * p] = p;
17         }
18     }
19 }
20 //分解质因数
21 vector<int> getPrimes(int x){
22     vector<int> res;
23     while(x > 1){
24         int p = min_primes[x];
25         res.push_back(p);
26         while(x % p == 0) x /= p;
27     }
28     return res;
29 }
30 //分解质因数带次数(幂){p, cnt}
31 vector<pair<int, int>> getPrimesCnt(int x){
32     vector<pair<int, int>> res;
33     while(x > 1){
34         int p = min_primes[x];
35         int cnt = 0;
36         while(x % p == 0){
37             x /= p;
38             cnt++;
39         }
40         res.push_back({p, cnt});
41     }
42     return res;
43 }
44
45
46 // 线性筛综合版: 质数表 + 最小质因子 + 莫比乌斯函数 + 欧拉函数 + 约数个数 + 约数和
47 // 调用 linear_sieve(maxn) 后直接使用全局数组
48 bool is_comp[maxn + 5];
49 int mu[maxn + 5]; // 莫比乌斯函数

```

```

50 int phi_arr[maxn + 5]; // 欧拉函数
51 int div_cnt[maxn + 5]; // 约数个数
52 int div_sum[maxn + 5]; // 约数和
53 int minp_exp[maxn + 5]; // 最小质因子的指数
54 int minp_pwsum[maxn + 5]; //  $1 + p + p^2 + \dots + p^{\text{最小质因子指数}}$ 
55
56 void linear_sieve(int n){
57     mu[1] = 1; phi_arr[1] = 1; div_cnt[1] = 1; div_sum[1] = 1;
58     for(int i = 2; i <= n; i++){
59         if(!is_comp[i]){
60             primes.push_back(i);
61             mu[i] = -1;
62             phi_arr[i] = i - 1;
63             div_cnt[i] = 2;
64             div_sum[i] = i + 1;
65             minp_exp[i] = 1;
66             minp_pwsum[i] = i + 1;
67         }
68         for(int p : primes){
69             int m = i * p;
70             if(m > n) break;
71             is_comp[m] = true;
72             if(i % p == 0){
73                 mu[m] = 0;
74                 phi_arr[m] = phi_arr[i] * p;
75                 minp_exp[m] = minp_exp[i] + 1;
76                 minp_pwsum[m] = minp_pwsum[i] * p + 1;
77                 div_cnt[m] = div_cnt[i] / (minp_exp[i] + 1) * (minp_exp[m] + 1);
78                 div_sum[m] = div_sum[i] / minp_pwsum[i] * minp_pwsum[m];
79                 break;
80             }else{
81                 mu[m] = -mu[i];
82                 phi_arr[m] = phi_arr[i] * (p - 1);
83                 minp_exp[m] = 1;
84                 minp_pwsum[m] = p + 1;
85                 div_cnt[m] = div_cnt[i] * 2;
86                 div_sum[m] = div_sum[i] * (p + 1);
87             }
88         }
89     }
90 }
91
92 // 旧欧拉筛单独函数
93 // void init(){
94 //     vector<bool> nop(maxn + 5);
95 //     nop[0] = nop[1] = true;
96 //     for(int i = 2; i <= maxn; i++){
97 //         if(!nop[i]){
98 //             primes.push_back(i);
99 //             if(i > sqrt(maxn)) continue;
100 //             for(int j = i * i; j <= maxn; j++){
101 //                 nop[j] = true;
102 //             }
103 //         }
104 //     }
105 // }
106 // }
107
108 int ksm(int x, int a, int m){
109     int ans = 1;
110

```

```

111     while(a){
112         if(a & 1) ans = ans * x % m;
113         x = x * x % m;
114         a >>= 1;
115     }
116     return ans;
117 }
118
119
120 //扩展欧几里得算法
121 int exgcd(int a, int b, int &x, int &y){
122     if(!b){
123         x = 1;
124         y = 0;
125         return a;
126     }
127     int d = exgcd(b, a % b, x, y);
128     int t = x;
129     x = y;
130     y = t - (a / b) * y;
131     return d; //最大公约数
132 }
133
134 //中国剩余定理( $x \equiv a \pmod{r}$ )
135 int CRT(int k, vector<int> a, vector<int> r){
136     int n = 1, ans = 0;
137     for(int i = 0; i < k; i++){
138         n = n * r[i];
139     }
140     for(int i = 0; i < k; i++){
141         int m = n / r[i], b, y;
142         exgcd(m, r[i], b, y);
143         ans = (ans + a[i] * m * b % n) % n;
144     }
145     return (ans % n + n) % n;
146 }
147
148 //扩展中国剩余定理
149 //求解方程组  $x \equiv a[i] \pmod{m[i]}$ 
150 int excrt(vector<int> m, vector<int> a, int n){
151     int M = m[0];
152     int ans = a[0];
153     for(int i = 1; i < n; i++){
154         int c = (a[i] - ans % m[i] + m[i]) % m[i];
155         int x, y;
156         int d = exgcd(M, m[i], x, y);
157         if(c % d) return -1;
158         x = x * (c / d) % (m[i] / d);
159         ans += x * M;
160         M = M / d * m[i];
161         ans = (ans % M + M) % M;
162     }
163     return ans;
164 }
165
166 //一个数的欧拉函数
167 int oula_phi(int n){
168     int ans = n;
169     for(int i = 2; i * i <= n; i++){
170         if(n % i == 0){
171             ans = ans / i * (i - 1);

```

```

172         while(n % i == 0) n /= i;
173     }
174 }
175 if(n > 1) ans = ans / n * (n - 1);
176 return ans;
177 }
178
179 //筛法求欧拉函数
180 vector<int> sieve_phi(int n) {
181     vector<int> phi(n + 1);
182     vector<bool> st(n + 1, false);
183     vector<int> primes;
184
185     phi[1] = 1; // 特殊情况: 1的欧拉函数值为1[1,3,5](@ref)
186
187     for (int i = 2; i <= n; i++) {
188         if (!st[i]) { // i是质数
189             primes.push_back(i);
190             phi[i] = i - 1; // 质数的欧拉函数值为i-1[1,2](@ref)
191         }
192
193         for (int j = 0; primes[j] <= n / i; j++) {
194             st[primes[j] * i] = true;
195
196             if (i % primes[j] == 0) {
197                 // primes[j]是i的最小质因子
198                 phi[primes[j] * i] = phi[i] * primes[j]; // 情况1[2,5](@ref)
199                 break;
200             } else {
201                 // primes[j]与i互质
202                 phi[primes[j] * i] = phi[i] * (primes[j] - 1); // 情况2[1,3](@ref)
203             }
204         }
205     }
206
207     return phi;
208 }
209
210 // 莫比乌斯反演
211 // 若  $g(n) = \sum_{d|n} f(d)$ , 则  $f(n) = \sum_{d|n} \mu(d) * g(n/d)$ 
212 // 若  $g(n) = \sum_{n|d} f(d)$ , 则  $f(n) = \sum_{n|d} \mu(d/n) * g(d)$ 
213 // 常用结论:
214 //  $[\gcd(i, j) == 1] = \sum_{d | \gcd(i, j)} \mu(d)$ 
215 //  $\phi(n) = \sum_{d|n} \mu(d) * (n / d)$ 
216 // 线性筛求  $\mu / \phi$  见上方 linear_sieve
217
218 // 狄利克雷前缀/后缀 变换 (快速 zeta 变换,  $O(n \log \log n)$ )
219 // 需要先调用 linear_sieve(maxn) 得到全局质数表 primes
220 // 注意: 变换是原地进行的, 若需保留原数组请先拷贝
221
222 // 前缀和: 变换后  $f[n] = \sum_{d|n} f[d]$ 
223 void dirichlet_prefix_sum(vector<int>& f, int n){
224     for(int p : primes)
225         for(int i = 1; i * p <= n; i++)
226             f[i * p] += f[i];
227 }
228
229 // 后缀和: 变换后  $f[d] = \sum_{d|n} f[n]$ 
230 void dirichlet_suffix_sum(vector<int>& f, int n){
231     for(int p : primes)
232         for(int i = n / p; i >= 1; i--)

```

```

233         f[i] += f[i * p];
234     }
235
236     // 前缀最值: 变换后  $f[n] = \max_{d|n} f[d]$ 
237     void dirichlet_prefix_max(vector<int>& f, int n){
238         for(int p : primes)
239             for(int i = 1; i * p <= n; i++)
240                 f[i * p] = max(f[i * p], f[i]);
241     }
242
243     // 前缀最值: 变换后  $f[n] = \min_{d|n} f[d]$ 
244     void dirichlet_prefix_min(vector<int>& f, int n){
245         for(int p : primes)
246             for(int i = 1; i * p <= n; i++)
247                 f[i * p] = min(f[i * p], f[i]);
248     }
249
250     // 前缀和的逆变换: 已知  $g[n] = \sum_{d|n} f[d]$ , 由 g 还原 f
251     void dirichlet_prefix_inv(vector<int>& f, int n){
252         for(int p : primes)
253             for(int i = n / p; i >= 1; i--)
254                 f[i * p] -= f[i];
255     }
256
257     // 后缀和的逆变换: 已知  $g[d] = \sum_{d|n} f[n]$ , 由 g 还原 f
258     void dirichlet_suffix_inv(vector<int>& f, int n){
259         for(int p : primes)
260             for(int i = 1; i * p <= n; i++)
261                 f[i] -= f[i * p];
262     }
263
264     // 狄利克雷卷积快速幂 - 计算f的k次卷积幂
265     void dirichlet_power(const vector<int>& f, vector<int>& res, int n, int k) {
266         // 初始化结果为狄利克雷卷积的单位元e
267         res.assign(n + 1, 0);
268         res[1] = 1; //  $\epsilon(1)=1, \epsilon(n)=0(n>1)$ 
269
270         vector<int> temp(f), base(f);
271
272         while (k) {
273             if (k & 1) {
274                 vector<int> new_res(n + 1, 0);
275                 // 计算  $res = res * base$ 
276                 for (int i = 1; i <= n; i++) {
277                     for (int j = 1; i * j <= n; j++) {
278                         new_res[i * j] = (new_res[i * j] + res[i] * base[j]) % mod;
279                     }
280                 }
281                 res = new_res;
282             }
283
284             // 计算  $base = base * base$ 
285             vector<int> new_base(n + 1, 0);
286             for (int i = 1; i <= n; i++) {
287                 for (int j = 1; i * j <= n; j++) {
288                     new_base[i * j] = (new_base[i * j] + base[i] * base[j]) % mod;
289                 }
290             }
291             base = new_base;
292
293             k >>= 1;

```

```

294     }
295 }
296
297 // 阶乘表，通常n不会太大，可以预先计算好
298 // 实际使用时可以根据需要动态计算或传入
299 const int fact[10] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
300
301 /**
302  * 康托展开，求排列的字典序排第几，O(n^2)
303  * @param permutation 排列，元素为0..n-1或1..n均可，但需是连续不重复的整数
304  * @return 康托展开值（即比当前排列小的排列个数）
305  */
306 int cantor_expand(const std::vector<int>& permutation) {
307     int n = permutation.size();
308     int code = 0;
309
310     for (int i = 0; i < n; ++i) {
311         int smaller = 0; // 统计未出现的元素中有多少个比当前元素小
312         for (int j = i + 1; j < n; ++j) {
313             if (permutation[j] < permutation[i]) {
314                 smaller++;
315             }
316         }
317         code += smaller * fact[n - i - 1];
318     }
319
320     return code;
321 }
322
323 /**
324  * 逆康托展开
325  * @param code 康托展开值（比目标排列小的排列个数）
326  * @param n 排列长度
327  * @param elements 可选的元素集合，默认使用0..n-1；如果是1..n则需传入对应集合
328  * @return 对应的排列
329  */
330 std::vector<int> cantor_expand_inv(int code, int n, const std::vector<int>&
elements = {}) {
331     std::vector<int> result;
332
333     // 确定可用的元素集合
334     std::vector<int> available;
335     if (elements.empty()) {
336         // 默认使用0..n-1
337         for (int i = 0; i < n; ++i) {
338             available.push_back(i);
339         }
340     } else {
341         available = elements;
342     }
343
344     for (int i = n; i > 0; --i) {
345         int fact_val = fact[i - 1];
346         int index = code / fact_val; // 确定应该取第几小的数
347         code %= fact_val; // 更新余数
348
349         result.push_back(available[index]);
350         available.erase(available.begin() + index);
351     }
352
353     return result;

```

```

354 }
355
356
357 // FFT 需要用 double, 所以局部不建议用全局的 int int 覆盖所有变量
358 struct Complex {
359     double r, i;
360     Complex(double r = 0, double i = 0) : r(r), i(i) {}
361     Complex operator + (const Complex& t) const { return {r + t.r, i + t.i}; }
362     Complex operator - (const Complex& t) const { return {r - t.r, i - t.i}; }
363     Complex operator * (const Complex& t) const { return {r * t.r - i * t.i, r *
t.i + i * t.r}; }
364 };
365
366 const double PI = acos(-1.0);
367 vector<int> rev; // 预处理位逆序
368
369 void fft_init(int len) {
370     rev.assign(len, 0);
371     for (int i = 0; i < len; i++)
372         rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (len >> 1) : 0);
373 }
374
375 void fft(vector<Complex>& a, int type) {
376     int n = a.size();
377     for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
378     for (int mid = 1; mid < n; mid <= 1) {
379         Complex wn(cos(PI / mid), type * sin(PI / mid));
380         for (int i = 0; i < n; i += (mid << 1)) {
381             Complex w(1, 0);
382             for (int j = 0; j < mid; j++, w = w * wn) {
383                 Complex x = a[i + j], y = w * a[i + j + mid];
384                 a[i + j] = x + y;
385                 a[i + j + mid] = x - y;
386             }
387         }
388     }
389     if (type == -1) for (auto& x : a) x.r /= n;
390 }
391
392 // 多项式乘法封装
393 vector<int> multiply_fft(vector<int>& A, vector<int>& B) {
394     int n = A.size(), m = B.size(), total = n + m - 1;
395     int len = 1; while (len < total) len <= 1;
396     fft_init(len);
397     vector<Complex> fa(len), fb(len);
398     for (int i = 0; i < n; i++) fa[i].r = A[i];
399     for (int i = 0; i < m; i++) fb[i].r = B[i];
400     fft(fa, 1); fft(fb, 1);
401     for (int i = 0; i < len; i++) fa[i] = fa[i] * fb[i];
402     fft(fa, -1);
403     vector<int> res(total);
404     for (int i = 0; i < total; i++) res[i] = (int)(fa[i].r + 0.5);
405     return res;
406 }
407
408 // // 假设输入两个多项式的系数
409 //     vector<int> a = {1, 2, 1}; // x^2 + 2x + 1
410 //     vector<int> b = {1, 1};    // x + 1
411
412 //     int n = a.size(), m = b.size();
413 //     int len = 1;

```



```

414 // while (len < n + m - 1) len <= 1; // 补齐到2的幂次
415
416 // vector<Complex> fa(len), fb(len);
417 // for (int i = 0; i < n; i++) fa[i] = Complex(a[i], 0);
418 // for (int i = 0; i < m; i++) fb[i] = Complex(b[i], 0);
419
420 // fft(fa, len, 1);
421 // fft(fb, len, 1);
422
423 // // 点值相乘
424 // for (int i = 0; i < len; i++) fa[i] = fa[i] * fb[i];
425
426 // fft(fa, len, -1);
427
428 // // 输出结果 (四舍五入)
429 // for (int i = 0; i < n + m - 1; i++) {
430 //     cout << (int)(fa[i].r + 0.5) << " ";
431 // }
432 // return 0;
433
434
435 // ===== NTT 快速数论变换 =====
436 // 前提: NTT 需要质数模数  $p = c \cdot 2^k + 1$  (NTT 友好模数), 变换长度只支持 2 的幂且  $\leq 2^k$ 。
437 // 本模板:  $\text{NTT\_MOD} = 998244353 = 119 \cdot 2^{23} + 1 \rightarrow$  长度上限  $2^{23} \approx 8.4e6$ 。
438 //
439 // ( $\text{NTT\_MOD}$ ,  $G$ ) 是一对 (模数, 原根), 两者的关系是 NTT 的核心:
440 // 1) 模  $p$  的乘法群  $(\mathbb{Z}/p\mathbb{Z})^*$  是循环群, 阶为  $p-1 = 119 \cdot 2^{23}$ ;
441 // 2)  $G = 3$  是该群的一个"原根"(生成元): 3 的阶恰好等于  $p-1$ 
442 //     (比  $p-1$  小的任何次幂都不等于 1), 所以 3 的各次幂能遍历全部非零元素;
443 // 3) 只要  $\text{len} \mid p-1$ , 则  $w_{\text{len}} = G^{((p-1)/\text{len})} \bmod p$  就是  $\text{len}$  次本原单位根:
444 //      $w_{\text{len}}^{\text{len}} = G^{(p-1)} \equiv 1$  (费马小定理), 且  $w_{\text{len}}$  的阶恰为  $\text{len}$ ,
445 //     这正是 NTT 蝶形运算每一层  $\text{len}$  所用的旋转因子 (见下方 ntt 里 wlen);
446 // 4) 逆变换需要逆单位根  $w_{\text{len}}^{-1} = (G^{-(p-1)/\text{len}})^{(p-1)/\text{len}}$ ,
447 //     因此定义  $GI = G^{-(p-1)} = 332748118$  ( $3 \cdot 332748118 \equiv 1 \bmod p$ );
448 // 5) 一句话总结: 模数决定"能变换多长"(看  $p-1$  里因子 2 的个数),
449 //     原根决定"具体用哪个单位根"(由原根的幂次生成)。
450 //
451 // 注意: 换模数必须同步换原根。常用 (模数, 原根) 组合 (3 是这些模数的公共原根, 非巧合):
452 //  $167772161 = 5 \cdot 2^{25} + 1 \rightarrow$  长度上限  $2^{25}$ 
453 //  $469762049 = 7 \cdot 2^{26} + 1 \rightarrow$  长度上限  $2^{26}$ 
454 //  $998244353 = 119 \cdot 2^{23} + 1 \rightarrow$  长度上限  $2^{23}$  (本模板)
455 //  $1004535809 = 479 \cdot 2^{21} + 1 \rightarrow$  长度上限  $2^{21}$ 
456 // 原根判定法: 对  $p-1$  的每个质因子  $q$ , 若  $g^{(p-1)/q} \equiv 1 \pmod p$ , 则  $g$  是原根。
457 // 与 FFT 相比: NTT 全程整数运算、无浮点误差, 但每个系数都会先对  $\text{NTT\_MOD}$  取模。
458 // 用法: 多项式系数按升幂存入 vector<int>, 直接调用 multiply_ntt(A, B) 即可。
459 // 依赖上方通用快速幂 ksm。
460 const int NTT_MOD = 998244353;
461 const int G = 3; // NTT_MOD 的原根 (生成元):  $w_{\text{len}} = G^{((p-1)/\text{len})}$ 
462 const int GI = 332748118; // G 的逆元: 逆变换用  $w_{\text{len}}^{-1} = GI^{((p-1)/\text{len})}$ 
463
464 // 位逆序表: ntt_rev[i] = i 按  $\log_2(n)$  位反转后的下标, 变换前按当前长度  $n$  更新
465 vector<int> ntt_rev;
466 void ntt_rev_init(int n) {
467     ntt_rev.resize(n);
468     for (int i = 0; i < n; i++)
469         ntt_rev[i] = (ntt_rev[i >> 1] >> 1) | ((i & 1) ? (n >> 1) : 0);
470 }
471
472 // 原地 NTT 变换 (蝶形运算)
473 // type = 1: 正变换, 系数表示  $\rightarrow$  点值表示
474 // type = -1: 逆变换, 点值表示  $\rightarrow$  系数表示 (结尾自动乘  $1/n$ )

```

```

475 // 前置条件: a.size() 为 2 的幂 (且  $\leq 2^{23}$ ), 并已按该长度调用过 ntt_rev_init
476 void ntt(vector<int>& a, int type) {
477     int n = a.size();
478     // 1. 位逆序重排, 把系数放到分治正确的位置
479     for (int i = 0; i < n; i++)
480         if (i < ntt_rev[i]) swap(a[i], a[ntt_rev[i]]);
481     // 2. 自底向上合并, len 为当前合并块的长度 (2, 4, 8, ...)
482     for (int len = 2; len <= n; len <= 1) {
483         int wlen = ksm(type == 1 ? G : GI, (NTT_MOD - 1) / len, NTT_MOD); // len
次单位根
484         for (int i = 0; i < n; i += len) { // 遍历每个块
485             int w = 1;
486             for (int j = 0; j < len / 2; j++, w = w * wlen % NTT_MOD) { // 块内蝶形
487                 int x = a[i + j]; // 左半边
488                 int y = w * a[i + j + len / 2] % NTT_MOD; // 右半边乘旋转因子
489                 a[i + j] = (x + y) % NTT_MOD;
490                 a[i + j + len / 2] = (x - y + NTT_MOD) % NTT_MOD;
491             }
492         }
493     }
494     if (type == -1) { // 3. 逆变换最后整体除以 n
495         int inv_n = ksm(n, NTT_MOD - 2, NTT_MOD);
496         for (int& x : a) x = x * inv_n % NTT_MOD;
497     }
498 }
499
500 // NTT 加速的多项式乘法, 返回系数均在 [0, NTT_MOD) 内的结果
501 // 返回 C, 长度 n + m - 1, 满足  $C[k] = \sum_{i+j=k} A[i] * B[j] \pmod{NTT\_MOD}$ 
502 // 限制:  $n + m - 1 \leq 2^{23}$ ; 超出需换大模数 (如 469762049, 1004535809) 或拆系数 NTT
503 // 使用示例: vector<int> C = multiply_ntt(A, B);
504 vector<int> multiply_ntt(const vector<int>& A, const vector<int>& B) {
505     if (A.empty() || B.empty()) return {}; // 空多项式
506     int n = A.size(), m = B.size(), total = n + m - 1;
507     int len = 1; // 结果长度为 total, 补到不小于它的 2
的幂
508     while (len < total) len <= 1;
509     vector<int> fa(A), fb(B); // 拷贝一份, 不修改入参
510     fa.resize(len); fb.resize(len);
511     ntt_rev_init(len);
512     ntt(fa, 1); ntt(fb, 1); // 系数 -> 点值
513     for (int i = 0; i < len; i++) fa[i] = fa[i] * fb[i] % NTT_MOD; // 点值相乘
514     ntt(fa, -1); // 点值 -> 系数
515     fa.resize(total); // 高次项系数均为 0, 截掉
516     return fa;
517 }
518
519 // 使用示例:
520 // vector<int> A = {1, 2, 1}; // A(x) = 1 + 2x + x^2
521 // vector<int> B = {1, 1}; // B(x) = 1 + x
522 // vector<int> C = multiply_ntt(A, B); // C = {1, 3, 3, 1}
523
524
525 // ===== 多项式求逆 (NTT 加速)
=====
526 // 求 B(x) 使得  $A(x) \cdot B(x) \equiv 1 \pmod{x^n}$ , 所有系数均取模 NTT_MOD (998244353)
527 // 依赖: ksm + ntt + ntt_rev_init + multiply_ntt
528 // 要求: A[0] != 0 (常数项在模意义下存在逆元)
529 // 原理: 牛顿迭代. 若 B 满足  $A \cdot B \equiv 1 \pmod{x^m}$ , 则
530 //  $B' = B \cdot (2 - A \cdot B)$  满足  $A \cdot B' \equiv 1 \pmod{x^{2m}}$ 
531 // 长度每次翻倍, 总复杂度  $O(n \log n)$ . len = 4m 保证卷积无循环串位
532 vector<int> poly_inv(const vector<int>& A, int n){

```

```

533 vector<int> B(1, ksm(A[0], NTT_MOD - 2, NTT_MOD)); // 初始 B0 = A[0]^{-1}
534 int m = 1; // 当前精度: 模 x^m
535 while(m < n){
536     int len = m << 2; // NTT 长度 4m
537     vector<int> a(min((int)A.size(), m << 1)), b(B); // A 截断到 2m 项
538     for(int i = 0; i < (int)a.size(); i++) a[i] = A[i];
539     a.resize(len); b.resize(len);
540     ntt_rev_init(len);
541     ntt(a, 1); ntt(b, 1);
542     for(int i = 0; i < len; i++) // 点值域: B(2 - A·B)
543         a[i] = b[i] * (2 - a[i] * b[i] % NTT_MOD + NTT_MOD) % NTT_MOD;
544     ntt(a, -1);
545     B.assign(a.begin(), a.begin() + (m << 1)); // 精度翻倍到 2m
546     m <=<= 1;
547 }
548 B.resize(n); // 截断到所需长度
549 return B;
550 }
551 // 使用示例:
552 // vector<int> A = {1, 1, 1}; // A(x) = 1 + x + x^2
553 // vector<int> B = poly_inv(A, 3); // B = {1, mod-1, 0}, 即 1 - x
554 // 验证: vector<int> C = multiply_ntt(A, B); C.resize(3); // C = {1, 0, 0}
555
556
557 // 快速沃尔什变换 FWT (与/或/异或 卷积, O(n log n), n 为 2 的幂, 模 mod 运算)
558 // 用法: a、b 长度补成 2 的幂 → 各自正变换 → 点乘 → 逆变换
559
560 // 或卷积: c[k] = Σ_{i|j==k} a[i]*b[j]
561 void fwt_or(vector<int>& a, bool inv){
562     int n = a.size();
563     for(int len = 1; len < n; len <=<= 1)
564         for(int i = 0; i < n; i += len << 1)
565             for(int j = 0; j < len; j++){
566                 int &x = a[i + j], &y = a[i + j + len];
567                 if(inv) y = (y - x + mod) % mod;
568                 else y = (y + x) % mod;
569             }
570 }
571
572 // 与卷积: c[k] = Σ_{i&j==k} a[i]*b[j]
573 void fwt_and(vector<int>& a, bool inv){
574     int n = a.size();
575     for(int len = 1; len < n; len <=<= 1)
576         for(int i = 0; i < n; i += len << 1)
577             for(int j = 0; j < len; j++){
578                 int &x = a[i + j], &y = a[i + j + len];
579                 if(inv) x = (x - y + mod) % mod;
580                 else x = (x + y) % mod;
581             }
582 }
583
584 // 异或卷积: c[k] = Σ_{i^j==k} a[i]*b[j]
585 void fwt_xor(vector<int>& a, bool inv){
586     int n = a.size();
587     for(int len = 1; len < n; len <=<= 1)
588         for(int i = 0; i < n; i += len << 1)
589             for(int j = 0; j < len; j++){
590                 int x = a[i + j], y = a[i + j + len];
591                 a[i + j] = (x + y) % mod;
592                 a[i + j + len] = (x - y + mod) % mod;
593             }

```

```

594     if(inv){
595         int in = ksm(n, mod - 2, mod);
596         for(int& v : a) v = v * in % mod;
597     }
598 }
599
600 // FWT 卷积封装, 传入 fwt_or / fwt_and / fwt_xor
601 vector<int> fwt_conv(vector<int> a, vector<int> b, void (*f)(vector<int>&, bool))
602 {
603     int n = 1, mx = max(a.size(), b.size());
604     while(n < mx) n <= 1;
605     a.resize(n); b.resize(n);
606     f(a, false); f(b, false);
607     for(int i = 0; i < n; i++) a[i] = a[i] * b[i] % mod;
608     f(a, true);
609     return a;
610 }
611 // 整除分块:  $\sum_{i=1}^n \lfloor n/i \rfloor$  取值只有  $O(\sqrt{n})$  段
612 // 每段  $[l, r]$  内  $\lfloor n/i \rfloor$  相同,  $r = n / (n / l)$ 
613 int div_block_sum(int n){
614     int ans = 0;
615     for(int l = 1, r; l <= n; l = r + 1){
616         r = n / (n / l);
617         ans += (n / l) * (r - l + 1);
618     }
619     return ans;
620 }
621 // 推广:  $\sum_{i=1}^{\min(n,m)} \lfloor n/i \rfloor * \lfloor m/i \rfloor$  同理按  $r = \min(n/(n/l), m/(m/l))$  分段
622
623
624 // ===== 杜教筛 =====
625 // 用途: 线性筛  $O(n)$  只能处理  $n \leq 1e7 \sim 1e8$ ; 杜教筛在  $O(n^{2/3})$  内求积性函数前缀和
626 //  $S(n) = \sum_{i=1}^n f(i)$ ,  $n$  可到  $1e9 \sim 1e10$  (更大需调阈值并注意内存)。
627 // 原理:
628 // 选  $g$  使  $h = f * g$  (狄利克雷卷积) 的前缀和  $H(n) = \sum_{i \leq n} h(i)$  能  $O(1)$  求, 由
629 //  $H(n) = \sum_{d=1}^n g(d) S(\lfloor n/d \rfloor)$  移项得
630 //  $S(n) = (H(n) - \sum_{d=2}^n g(d) S(\lfloor n/d \rfloor)) / g(1)$ 
631 // 右边  $\sum$  用整除分块压成  $O(\sqrt{n})$  段; 小值  $v \leq T$  由线性筛预处理。
632 // 常见配对:  $(\mu, 1) \rightarrow h = \epsilon, H(n) = 1$  (梅滕斯函数)
633 //  $(\phi, 1) \rightarrow h = id, H(n) = n(n+1)/2$ 
634 //  $(\mu \cdot id^k, id^k) \rightarrow h = \epsilon$ 
635 // 记忆化不需要哈希表: 递归查询的键全是  $\lfloor N/i \rfloor$  形式 ( $\lfloor \lfloor N/a \rfloor / b \rfloor = \lfloor N/(ab) \rfloor$ ),
636 // 只有  $2\sqrt{N}$  个不同值; 键  $v \leq T$  查线性筛前缀, 键  $v > T$  存 memo[N/V] ( $N/V < N/T$ , 下标安全)。
637 // 注意"大键换小下标": 进记忆化分支的键必然  $v > T$ , 所以下标  $N/v < N/T$  很小;
638 // 例如  $N=1e9, T \approx 1e6$  时,  $v=2$  直接查 smu[2], 大键  $v=1e9/5e8$  的下标只有 1/2,
639 // 数组只开  $N/T+2 \approx 1002$  个元素, 绝不按  $N$  或按  $v$  开。
640 // 注意用 vis 标记"已算过"而不是看值是否为 0 —  $\mu$  前缀和可以真的等于 0!
641 // 复杂度: 阈值  $T = n^{2/3}$  时  $O(n^{2/3})$ ;  $T$  越大预处理越重、递归越轻。
642 // 使用示例:
643 // Dujiao dj; dj.init(1e9);
644 // auto [smu, sphi] = dj.calc(1e9); // smu =  $\sum_{i \leq 1e9} \mu(i) \bmod 1e9+7$ , sphi 同理
645 // 若要求精确值(不取模): 去掉 calc 中所有 % mod 即可 (注意  $\phi$  前缀和  $n$  大时会超 long long)。
646 struct Dujiao {
647     long long N; // 目标上界
648     long long T; // 阈值: 线性筛预处理到 T, 键  $v > T$  走记忆化

```

```

649     vector<int32_t> mu, phi, smu, sph;           // 小范围前缀 (int32 省内存)
650     vector<long long> memo_mu, memo_phi;       // 大键记忆化: 下标 N/v
651     vector<char> vis;                          // 已算标记 (不能拿值==0 判断!)
652
653     void init(long long n){
654         N = n;
655         T = min((long long)pow((long double)N, 2.0 / 3.0) + 1, 5000000LL); //
n^{2/3}, 上限 5e6
656         if(T > N) T = N;                      // N 很小时直接全筛 其实不要也可以, 神金ds
非要保护 N=1 少开一个字节空间
657         mu.assign(T + 1, 0); phi.assign(T + 1, 0);
658         smu.assign(T + 1, 0); sph.assign(T + 1, 0);
659         vector<int32_t> nop(T + 1, 0);
660         vector<long long> pr;
661         mu[1] = 1; phi[1] = 1;
662         for(long long i = 2; i <= T; i++){
663             if(!nop[i]){ pr.push_back(i); mu[i] = -1; phi[i] = i - 1; }
664             for(long long p : pr){
665                 long long t = i * p;
666                 if(t > T) break;
667                 nop[t] = 1;
668                 if(i % p == 0){ mu[t] = 0; phi[t] = phi[i] * p; break; }
669                 mu[t] = -mu[i];
670                 phi[t] = phi[i] * (p - 1);
671             }
672         }
673         for(long long i = 1; i <= T; i++){
674             smu[i] = ((long long)smu[i - 1] + mu[i] + mod) % mod;
675             sph[i] = (sph[i - 1] + phi[i]) % mod;
676         }
677         long long sz = N / T + 2;              // 键 v > T ⇒ N/v < N/T, 数组只开 N/T
大小
678         memo_mu.assign(sz, 0); memo_phi.assign(sz, 0); vis.assign(sz, 0);
679     }
680
681     // 返回 (Σ_{i=1}^v μ(i), Σ_{i=1}^v φ(i)) 均对 mod 取模
682     pair<long long, long long> calc(long long v){
683         if(v <= T) return {smu[v], sph[v]};
684         long long idx = N / v;                // 大键的下标变换, O(1) 存取
685         if(vis[idx]) return {memo_mu[idx], memo_phi[idx]};
686         vis[idx] = 1;
687         long long m = 1;                      // S_μ(v) = 1 - Σ_{d=2..v} S_μ(v/d)
688         long long p = (long long)((__int128)v * (v + 1) / 2 % mod); // S_φ(v) =
v(v+1)/2 - Σ ...
689         for(long long l = 2; l <= v; ){
690             long long q = v / l, r = v / q; // 整除分块: [l,r] 内 [v/d] 都等于 q
691             long long len = (r - l + 1) % mod;
692             auto [sm, sp] = calc(q);
693             m = (m - len * sm % mod + mod) % mod;
694             p = (p - len * sp % mod + mod) % mod;
695             l = r + 1;
696         }
697         memo_mu[idx] = m; memo_phi[idx] = p;
698         return {m, p};
699     }
700 };
701 // 验证过的取值: M(10^6)=212, M(10^7)=1037, M(10^8)=1928, M(10^9)=-222,
M(10^10)=-33722;
702 // Σ_{i≤10^6} φ(i)=303963552392, Σ_{i≤10^7} φ(i)=30396356427242
703 // 实测耗时: n=1e9 约 27ms, n=1e10 约 135ms
704

```

```

705
706 // 类欧几里得算法 log求和 sum = i(0 - n-1) floor [(a*i + b) / c]
707 // 求i从0到n-1的ai+b/c向下取整的总和
708 // n >= 0, m > 0, a >= 0, b >= 0
709 int floor_sum(int n, int m, int a, int b) {
710     int ans = 0;
711     if (a >= m) {
712         ans += (n - 1) * n / 2 * (a / m);
713         a %= m;
714     }
715     if (b >= m) {
716         ans += n * (b / m);
717         b %= m;
718     }
719
720     int y_max = (a * n + b) / m;
721     int x_max = (y_max * m - b);
722     if (y_max == 0) return ans;
723
724     ans += (n - (x_max + a - 1) / a) * y_max;
725     ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
726     return ans;
727 }
728
729 //SOSDP
730 int n = 50;
731 int total_states = (1 << n);
732 vector<int> dp(total_states, 0); // dp数组初始化, 根据题意可能初始为A[i]
733
734 // SOS DP 计算超集和
735 for (int i = 0; i < n; ++i) {
736     for (int j = 0; j < total_states; ++j) {
737         if ((j >> i) & 1) { // 如果状态j的第i位是1
738             dp[j] += dp[j ^ (1 << i)]; // 累加不含第i位的状态的值
739         }
740     }
741 }
742
743 // 此时dp[mask]存储了所有满足 (i & mask) == mask 的A[i]之和

```

数论进阶

```

1 // ===== 数论进阶板子 =====
2 // 内容: 快速乘、Miller-Rabin、Pollard-Rho、欧拉降幂、BSGS/exBSGS、
3 //       Lucas/exLucas、阶与原根、二次剩余(Cipolla)、Min_25 筛
4 // 与 数论.cpp 合并时注意: 本文件自带 ksm / exgcd / oula_phi 副本, 请去重。
5 // 说明: 本文件 ksm 内部用 mul(龟速防溢出) 实现, 任何模数(含 2^64 级)都安全;
6 //       mul 基于 __int128, 若编译器不支持可换成龟速乘(见注释)。
7
8 const int mod = 1e9 + 7;
9
10 // ===== 基础工具 =====
11 // 模乘防溢出: 仅在 a,b < m 时保证正确
12 int mul(int a, int b, int m){
13     return (int)((__int128)a * b % m);
14 }
15
16 int ksm(int x, int a, int m){
17     int ans = 1;
18     x %= m; // 兼容大底数(如 Miller-Rabin 的 a)

```

```

19     while(a){
20         if(a & 1) ans = mul(ans, x, m);
21         x = mul(x, x, m);
22         a >>= 1;
23     }
24     return ans;
25 }
26
27 int exgcd(int a, int b, int &x, int &y){
28     if(!b){
29         x = 1;
30         y = 0;
31         return a;
32     }
33     int d = exgcd(b, a % b, x, y);
34     int t = x;
35     x = y;
36     y = t - (a / b) * y;
37     return d;
38 }
39
40 // 模逆元(要求 gcd(a,m)=1, m 不必为质数)
41 int inv(int a, int m){
42     int x, y;
43     exgcd(a, m, x, y);
44     return (x % m + m) % m;
45 }
46
47 // 单个数欧拉函数
48 int oula_phi(int n){
49     int ans = n;
50     for(int i = 2; i * i <= n; i++){
51         if(n % i == 0){
52             ans = ans / i * (i - 1);
53             while(n % i == 0) n /= i;
54         }
55     }
56     if(n > 1) ans = ans / n * (n - 1);
57     return ans;
58 }
59
60 // ===== Miller-Rabin 素性测试
61 // =====
62 // 费马测试 + 二次探测。底数集为  $2^{64}$  内的确定性集合(无需随机)。
63 // 复杂度  $O(k \log n)$ , k 为底数个数。
64 // 验证: n=341550071728321(伪素数) 判合数; n=998244353 判素数。
65 bool is_prime(int n){
66     if(n < 2) return false;
67     for(int p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){
68         if(n % p == 0) return n == p;
69     }
70     int d = n - 1, s = 0;
71     while(d % 2 == 0){
72         d /= 2;
73         s++;
74     }
75     for(int a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}){
76         if(a % n == 0) continue;
77         int x = ksm(a, d, n);
78         if(x == 1 || x == n - 1) continue;
79         bool ok = false;

```

```

79         for(int i = 1; i < s; i++){
80             x = mul(x, x, n);
81             if(x == n - 1){
82                 ok = true;
83                 break;
84             }
85         }
86         if(!ok) return false;
87     }
88     return true;
89 }
90
91 // ===== Pollard-Rho 大数分解
92 // =====
93 // 生日悖论：在模 q(待找因子) 的剩余系里取  $O(\sqrt{q})$  个伪随机数  $x_i = f(x_{i-1})$ ,
94 // 出现  $x_i \equiv x_j \pmod{q}$  时  $\gcd(|x_i - x_j|, n)$  即非平凡因子。
95 // Floyd 判圈：龟走 1 步、兔走 2 步,  $O(1)$  空间。(Brent 版更快, 需要时可替换)
96 // 期望复杂度  $O(n^{1/4})$ 。依赖 is_prime。
97 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
98
99 int pollard_rho(int n){
100     if(n % 2 == 0) return 2;
101     if(n % 3 == 0) return 3;
102     while(true){
103         int c = rng() % (n - 1) + 1;
104         int x = rng() % (n - 1) + 1, y = x;
105         for(;;){
106             x = (mul(x, x, n) + c) % n; // 龟
107             y = (mul(y, y, n) + c) % n; // 兔 x2
108             y = (mul(y, y, n) + c) % n;
109             int d = __gcd(abs(x - y), n);
110             if(d == n) break; // 环撞满, 换参数重试
111             if(d > 1) return d;
112         }
113     }
114 }
115
116 // 质因数分解(带指数), 结果存入 mp。使用示例:
117 // map<int,int> mp; factor(341550071728321, mp);
118 // mp = {10670053: 1, 32010157: 1}
119 void factor(int n, map<int,int>& mp){
120     if(n < 2) return;
121     if(is_prime(n)){
122         mp[n]++;
123         return;
124     }
125     int d = pollard_rho(n);
126     factor(d, mp);
127     factor(n / d, mp);
128 }
129
130 // ===== 欧拉降幂(扩展欧拉定理)
131 // =====
132 //  $a^b \equiv a^{b \bmod \phi(p)}$  ( $b < \phi(p)$ );  $a^b \equiv a^{(b \bmod \phi(p) + \phi(p))} \pmod{p}$  ( $b \geq \phi(p)$ )
133 // b 以字符串传入, 可超过 long long。依赖 oula_phi。
134 // 验证:  $2^{100} \bmod 7 = 2$  ( $\phi(7)=6$ ,  $100 \geq 6 \rightarrow 2^{(4+6)}=2^{10} \equiv 2$ )。
135 int euler_pow(int a, string b, int p){
136     int phi = oula_phi(p);
137     int e = 0;
138     bool big = false;
139     for(char c : b){

```



```

138     e = e * 10 + (c - '0');
139     if(e >= phi){
140         e %= phi;
141         big = true;
142     }
143 }
144 if(big) e += phi;
145 return ksm(a, e, p);
146 }
147
148 // ===== BSGS 离散对数 =====
149 // 求  $a^x \equiv b \pmod{p}$  的最小非负整数解  $x$ , 要求  $\gcd(a, p) = 1$ 。
150 // 大步小步:  $x = i*m - j$ ,  $m = \lceil \sqrt{p} \rceil$ , 哈希表存  $b * a^j$ 。
151 // 复杂度  $O(\sqrt{p})$ 。无解返回 -1。
152 // 验证:  $\text{bsgs}(2, 3, 5) = 3$  ( $2^3 \equiv 3 \pmod{5}$ )。
153 int bsgs(int a, int b, int p){
154     a %= p;
155     b %= p;
156     if(b == 1 || p == 1) return 0;
157     int m = ceil(sqrt(p));
158     unordered_map<int, int> mp;
159     int cur = b;
160     for(int j = 0; j < m; j++){ // baby step:  $b * a^j$ 
161         if(!mp.count(cur)) mp[cur] = j;
162         cur = cur * a % p;
163     }
164     int step = ksm(a, m, p); // giant step 步长  $a^m$ 
165     cur = 1;
166     for(int i = 0; i <= m; i++){
167         if(mp.count(cur)){
168             int x = i * m - mp[cur];
169             if(x >= 0) return x;
170         }
171         cur = cur * step % p;
172     }
173     return -1;
174 }
175
176 // 扩展 BSGS:  $\gcd(a, p)$  任意。逐次消公因子归约到普通 BSGS。
177 // 验证:  $\text{exbsgs}(2, 4, 6) = 2$ ;  $\text{exbsgs}(4, 7, 10) = -1$ 。
178 int exbsgs(int a, int b, int p){
179     a %= p;
180     b %= p;
181     if(b == 1 || p == 1) return 0;
182     int d = __gcd(a, p), k = 0, base = 1;
183     while(d != 1){
184         if(b % d) return -1; // b 不含因子 d, 且  $b \neq 1 \rightarrow$  无解
185         b /= d;
186         p /= d;
187         k++;
188         base = base * (a / d) % p;
189         if(base == b) return k; // 消到只剩常数的特判
190         d = __gcd(a, p);
191     }
192     int t = bsgs(a, b * inv(base, p) % p, p);
193     return t == -1 ? -1 : t + k;
194 }
195
196 // ===== Lucas 定理 / exLucas =====
197 // Lucas:  $C(n, m) \pmod{p}$ ,  $p$  为小质数 ( $n$  可远大于  $p$ )。

```

```

198 // 定理：把  $n, m$  按  $p$  进制展开,  $C(n, m) \equiv \prod C(n_i, m_i) \pmod p$ 。
199 // 验证:  $\text{lucas}(5, 2, 3) = 1$  ( $C(5, 2) = 10 \equiv 1$ ;  $5 = 12_3, 2 = 02_3 \rightarrow C(1, 0)C(2, 2) = 1$ )。
200 int lucas_fac[1000005];
201
202 void lucas_init(int p){
203     lucas_fac[0] = 1;
204     for(int i = 1; i < p; i++){
205         lucas_fac[i] = lucas_fac[i - 1] * i % p;
206     }
207 }
208
209 //  $C(n, m) \pmod p, n, m < p$ 
210 int C_small(int n, int m, int p){
211     if(m > n) return 0;
212     return lucas_fac[n] * ksm(lucas_fac[m], p - 2, p) % p
213         * ksm(lucas_fac[n - m], p - 2, p) % p;
214 }
215
216 int lucas(int n, int m, int p){
217     if(m == 0) return 1;
218     return C_small(n % p, m % p, p) * lucas(n / p, m / p, p) % p;
219 }
220
221 // exLucas:  $C(n, m) \pmod{p^k}$  ( $p$  质数)。核心：消  $p$  因子 + 周期阶乘 + 逆元。
222 //  $\text{fac\_ex}[i] = \prod_{j \leq i, p \nmid j} j \pmod{pk}$ , 每  $pk$  一个周期。
223 int fac_ex[1000005];
224
225 void exlucas_init(int p, int pk){
226     fac_ex[0] = 1;
227     for(int i = 1; i <= pk; i++){
228         if(i % p) fac_ex[i] = fac_ex[i - 1] * i % pk;
229         else fac_ex[i] = fac_ex[i - 1];
230     }
231 }
232
233 //  $n!$  除去全部  $p$  因子后的部分  $\pmod{pk}$ 
234 int f(int n, int p, int pk){
235     if(n == 0) return 1;
236     return f(n / p, p, pk) * ksm(fac_ex[pk], n / pk, pk) % pk * fac_ex[n % pk] %
pk;
237 }
238
239 //  $C(n, m) \pmod{p^k}$ 
240 int C_mod_pk(int n, int m, int p, int k){
241     int pk = 1;
242     for(int i = 0; i < k; i++) pk *= p;
243     int cnt = 0; //  $v_p(C(n, m))$ 
244     for(int i = n; i; i /= p) cnt += i / p;
245     for(int i = m; i; i /= p) cnt -= i / p;
246     for(int i = n - m; i; i /= p) cnt -= i / p;
247     if(cnt >= k) return 0; //  $p$  的指数超过  $pk$  中含量  $\rightarrow 0$ 
248     int res = 1;
249     if(cnt) res = res * ksm(p, cnt, pk) % pk;
250     res = res * f(n, p, pk) % pk;
251     res = res * inv(f(m, p, pk), pk) % pk;
252     res = res * inv(f(n - m, p, pk), pk) % pk;
253     return res;
254 }
255
256 //  $C(n, m) \pmod M, M$  任意合数：分解  $M = \prod p^k$ , 逐块求值再 CRT 合并。
257 // 验证:  $\text{exlucas}(5, 2, 12) = 10$  ( $\pmod 4$  得 2,  $\pmod 3$  得 1, CRT 合并得 10)。

```

```

258 int exlucas(int n, int m, int M){
259     int ans = 0, MM = M;
260     for(int p = 2; p * p <= MM; p++){
261         if(MM % p) continue;
262         int k = 0, pk = 1;
263         while(MM % p == 0){
264             MM /= p;
265             k++;
266             pk *= p;
267         }
268         exlucas_init(p, pk);
269         int a = C_mod_pk(n, m, p, k);
270         ans = (ans + a * (M / pk) % M * inv(M / pk % pk, pk) % M) % M;
271     }
272     if(MM > 1){
273         int p = MM, k = 1, pk = p;
274         exlucas_init(p, pk);
275         int a = C_mod_pk(n, m, p, k);
276         ans = (ans + a * (M / pk) % M * inv(M / pk % pk, pk) % M) % M;
277     }
278     return ans;
279 }
280
281 // ===== 阶与原根 =====
282 // 阶: 最小的  $k > 0$  使  $a^k \equiv 1 \pmod{p}$ 。存在性要求  $\gcd(a, p) = 1$ 。
283 // 求  $p$  的最小原根( $p$  为质数): 对  $p-1$  的每个质因子  $q$  检验  $g^{(p-1)/q} \not\equiv 1 \pmod{p}$ 。
284 // 验证:  $\text{primitive\_root}(998244353) = 3$ 。
285 int primitive_root(int p){
286     int phi = p - 1, x = phi;
287     vector<int> facs;
288     for(int i = 2; i * i <= x; i++){
289         if(x % i == 0){
290             facs.push_back(i);
291             while(x % i == 0) x /= i;
292         }
293     }
294     if(x > 1) facs.push_back(x);
295     for(int g = 2; ; g++){
296         bool ok = true;
297         for(int q : facs){
298             if(ksm(g, phi / q, p) == 1){
299                 ok = false;
300                 break;
301             }
302         }
303         if(ok) return g;
304     }
305 }
306
307 // ===== 二次剩余(Cipolla) =====
308 // 解  $x^2 \equiv n \pmod{p}$ ,  $p$  为奇质数。欧拉判别法判有解, 扩域  $\mathbb{F}_p[\sqrt{w}]$  上快速幂。
309 // 返回一个解  $x$ (另一个为  $p-x$ ), 无解返回  $-1$ 。
310 // 验证:  $\text{cipolla}(2, 7) = 4$  ( $4^2 \equiv 2 \pmod{7}$ )。
311 int cipolla(int n, int p){
312     n %= p;
313     if(n == 0) return 0;
314     if(ksm(n, (p - 1) / 2, p) != 1) return -1; // 勒让德符号 = -1, 非二次剩余
315     int a = 0, w = 0;
316     while(true){
317         a = rng() % p;
318         w = ((mul(a, a, p) - n) % p + p) % p; //  $w = a^2 - n$ 

```

```

319         if(ksm(w, (p - 1) / 2, p) == p - 1) break; // 选到非二次剩余的 w
320     }
321     // 计算  $(a + \sqrt{w})^{(p+1)/2}$ , 答案必为纯实数部分
322     pair<int,int> pw = {a, 1}, res = {1, 0}; // (x, y) 表示  $x + y\sqrt{w}$ 
323     auto mul2 = [&](pair<int,int> u, pair<int,int> v){
324         return make_pair((mul(u.first, v.first, p) + mul(mul(u.second, v.second,
325 p), w, p)) % p,
326 (mul(u.first, v.second, p) + mul(u.second, v.first, p))
327 % p);
328     };
329     int e = (p + 1) / 2;
330     while(e){
331         if(e & 1) res = mul2(res, pw);
332         pw = mul2(pw, pw);
333         e >>= 1;
334     }
335     return res.first;
336 }
337 // ===== Min_25 筛 =====
338 // 求  $\sum_{i=1}^n f(i)$ ,  $f$  积性且素数处  $f(p)$  为  $p$  的多项式组合(本模板  $f(p)=p(p-1)$ ,
339 // 即  $f(p)=p^2-p$ ,  $f(p^e)=p^e(p^e-1)$ ,  $f(1)=1$  — 洛谷 P5325)。
340 //  $n$  可达  $1e11$ , 复杂度  $O(n^{3/4}/\log n)$ 。
341 // 原理: 按最小质因子分类。 $G$  函数在  $[n/i]$  上递推筛出素数贡献,  $S$  递归枚举合数。
342 // 注意: 大数组用 int32_t 避免被 #define int long long 撑爆内存。
343 // 验证:  $\text{solve}(10) = 263$  (手算:  $f(1..10)=1, 2, 6, 12, 20, 12, 42, 56, 72, 40$ )。
344 const int MIN25_N = 1000005; //  $\geq 2\sqrt{n}$ 
345 namespace min25 {
346     long long n, sq;
347     int tot;
348     long long w[2 * MIN25_N]; //  $[n/i]$  的全部取值
349     int32_t g1[2 * MIN25_N], g2[2 * MIN25_N]; // 素数处  $\sum p, \sum p^2$ 
350     int32_t id1[MIN25_N], id2[MIN25_N];
351     vector<int> primes;
352     bool is_comp[MIN25_N];
353     int sp1[MIN25_N], sp2[MIN25_N]; // 素数前缀和(按素数个数索引)
354
355     int idx(long long v){ //  $[n/i] \rightarrow$  离散化下标
356         return v <= sq ? id1[v] : id2[n / v];
357     }
358
359     // 最小质因子  $\geq \text{primes}[y]$  (0-based 索引) 的所有数的  $f$  之和
360     // 注意: 递归可能传入  $y == \text{primes.size()}$  (表示"大于  $\sqrt{n}$  的素数"),
361     // 此时素数部分  $g - \text{sp}[\text{size}]$  恰为  $(\sqrt{n}, x]$  内的素数贡献。
362     int S(long long x, int y){
363         if(x <= 1) return 0;
364         if(y < (int)primes.size() && primes[y] > x) return 0;
365         if(y >= (int)primes.size() && primes.back() > x) return 0;
366         int k = idx(x);
367         // 素数贡献 = 全部素数的  $(p^2-p)$  - 前  $y$  个素数的  $(p^2-p)$ 
368         int ans = (g2[k] - g1[k] - (sp2[y] - sp1[y]) % mod + 2LL * mod) % mod;
369         for(int i = y; i < (int)primes.size() && 1LL * primes[i] * primes[i] <=
370 x; i++){
371             long long pe = primes[i];
372             for(int e = 1; pe <= x; e++, pe *= primes[i]){ // 枚举  $p^e \leq x$  (含最高
373 次幂)
374                 int val = pe % mod * ((pe - 1) % mod) % mod; //  $f(p^e) =$ 
375  $p^e(p^e-1)$ 
376                 ans = (ans + val * (S(x / pe, i + 1) + (e > 1))) % mod % mod;
377             }
378         }
379     }

```

```

375     }
376     return ans;
377 }
378
379 int solve(long long N){
380     n = N;
381     sq = sqrt(n);
382     memset(is_comp, 0, sizeof(is_comp));    // 支持多次调用：重置筛状态
383     primes.clear();
384     for(int i = 2; i <= sq; i++){
385         if(!is_comp[i]) primes.push_back(i);
386         for(int p : primes){
387             if(1LL * i * p > sq) break;
388             is_comp[i * p] = true;
389             if(i % p == 0) break;
390         }
391     }
392     for(int i = 1; i <= (int)primes.size(); i++){
393         sp1[i] = (sp1[i - 1] + primes[i - 1]) % mod;
394         sp2[i] = (sp2[i - 1] + 1LL * primes[i - 1] * primes[i - 1] % mod) %
mod;
395     }
396     // 离散化 [n/i], 初始按"全是素数"算 g1=Σp、g2=Σp^2
397     tot = 0;
398     int inv6 = ksm(6, mod - 2, mod);
399     for(long long l = 1, r; l <= n; l = r + 1){
400         r = n / (n / l);
401         w[++tot] = n / l;
402         long long v = w[tot] % mod;
403         g1[tot] = (v * (v + 1) / 2 % mod - 1 + mod) % mod;
404         g2[tot] = (v * (v + 1) % mod * (2 * v + 1) % mod * inv6 % mod - 1 +
mod) % mod;
405         if(w[tot] <= sq) id1[w[tot]] = tot;
406         else id2[n / w[tot]] = tot;
407     }
408     // 逐素数筛掉合数
409     for(int i = 0; i < (int)primes.size(); i++){
410         int p = primes[i];
411         for(int j = 1; j <= tot && 1LL * p * p <= w[j]; j++){
412             int k = idx(w[j] / p);
413             g1[j] = (g1[j] - 1LL * p * (g1[k] - sp1[i]) % mod + mod) % mod;
414             g2[j] = (g2[j] - 1LL * p * p % mod * (g2[k] - sp2[i]) % mod +
mod) % mod;
415         }
416     }
417     return (S(n, 0) + 1) % mod;    // +f(1)
418 }
419 }

```

线性基与矩阵

```

1  const int mod = 1e9 + 7;
2
3  //线性基LinearBasis
4  vector<int> p(64);
5  bool insert(int x){
6      for(int i = 63; i >= 0; i--){
7          if(!(x >> i)) continue;
8          if(!p[i]){
9              p[i] = x;

```

```

10         return true;
11     }
12     x ^= p[i];
13 }
14 return false;
15 }
16
17 //矩阵 (0-based)
18 struct mat{
19     int n, m;
20     vector<vector<int>> a;
21     mat(int x = 0, int y = 0){
22         n = x;
23         m = y;
24         a.resize(x, vector<int> (y));
25     }
26
27     mat operator+ (const mat& b){
28         mat c(n, m);
29         for(int i = 0; i < n; i++){
30             for(int j = 0; j < m; j++){
31                 c.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
32             }
33         }
34         return c;
35     }
36
37     mat operator- (const mat& b){
38         mat c(n, m);
39         for(int i = 0; i < n; i++){
40             for(int j = 0; j < m; j++){
41                 c.a[i][j] = ((a[i][j] - b.a[i][j]) % mod + mod) % mod;
42             }
43         }
44         return c;
45     }
46
47     mat operator* (const mat& b){
48         mat c(n, b.m);
49         for(int i = 0; i < n; i++){
50             for(int j = 0; j < b.m; j++){
51                 for(int k = 0; k < m; k++){
52                     c.a[i][j] = (c.a[i][j] + a[i][k] * b.a[k][j]) % mod;
53                 }
54             }
55         }
56         return c;
57     }
58
59     mat transpose(){
60         mat res(m, n);
61         for(int i = 0; i < n; i++){
62             for(int j = 0; j < m; j++){
63                 res.a[j][i] = a[i][j];
64             }
65         }
66         return res;
67     }
68
69     void print(){
70         for(int i = 0; i < n; i++){

```

```
71         for(int j = 0; j < m; j++){
72             cout << a[i][j] << " ";
73         }
74         cout << '\n';
75     }
76 }
77
78 };
79
80 mat ksm(mat x, int a){
81     int n = x.a.size();
82     mat res(n, n);
83     for(int i = 0; i < n; i++){
84         res.a[i][i] = 1;
85     }
86     while(a){
87         if(a & 1) res = res * x;
88         x = x * x;
89         a >>= 1;
90     }
91     return res;
92 }
```